

Numerical Methods Project

Interpolation, Data Analysis, and Numerical Integration

Name: Arad Shafiee

Student ID: 40312029

Course: Mathematical Software

Date: July 2025 (Tir 1405)

Contents

1	Project 1: Interpolation Methods	5
1.1	Introduction	5
1.2	Data Points	5
1.3	Lagrange Interpolation	5
1.4	Cubic Spline Interpolation	5
1.5	Visualization	6
1.6	Analysis	6
2	Project 2: Excel Data Analysis	7
2.1	Introduction	7
2.2	Data Description	7
2.3	Bar Chart	7
2.4	Statistical Analysis	7
2.5	Analysis	8
3	Project 3: Numerical Integration	9
3.1	Introduction	9
3.2	Mathematical Background	9
3.3	Trapezoidal Rule	9
3.4	Simpson's Rule	9
3.5	Gaussian Quadrature	9
3.6	Results	9
3.7	Convergence Analysis	10
3.8	Analysis	10
4	Conclusion	11
4.1	Interpolation Methods	11
4.2	Data Analysis	11
4.3	Numerical Integration	11
5	GitHub Repository	11
6	References	11

List of Figures

1	Comparison of Lagrange and Cubic Spline Interpolation Methods	6
2	Comparison of Algorithm Run Times (Bar Chart)	7
3	Convergence Comparison of Integration Methods	10

List of Tables

1	Given Data Points	5
2	Cubic Spline Coefficients	5
3	Algorithm Runtime Data	7
4	Statistical Summary	8
5	Comparison of Numerical Integration Methods	9

1 Project 1: Interpolation Methods

1.1 Introduction

This section implements two interpolation methods: Lagrange Interpolation and Cubic Spline Interpolation. These methods approximate functions passing through given discrete data points.

1.2 Data Points

The given data points are shown in Table 1.

Table 1: Given Data Points

x	1	2	3	4	5	6
y	1	3	5	8	5	2

1.3 Lagrange Interpolation

The Lagrange interpolation polynomial is defined as:

$$P(x) = \sum_{i=0}^n y_i L_i(x)$$

where the Lagrange basis polynomials are given by:

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

The resulting polynomial of degree 5 is:

$$P(x) = -0.7083x^5 + 9.8333x^4 - 50.8333x^3 + 121.1667x^2 - 131.4583x + 53$$

1.4 Cubic Spline Interpolation

The natural cubic spline uses piecewise cubic polynomials defined on each interval:

$$S_i(x) = a_i(x - x_i)^3 + b_i(x - x_i)^2 + c_i(x - x_i) + d_i, \quad x \in [x_i, x_{i+1}]$$

The coefficients for each interval are shown in Table 2.

Table 2: Cubic Spline Coefficients

Interval	a_i	b_i	c_i	d_i
[1, 2]	1.0000	2.1866	0.0000	-0.1866
[2, 3]	3.0000	1.6268	-0.5598	0.9330
[3, 4]	5.0000	3.3062	2.2392	-2.5455
[4, 5]	8.0000	0.1483	-5.3971	2.2488
[5, 6]	5.0000	-3.8995	1.3493	-0.4498

1.5 Visualization

Figure 1 shows the comparison of both interpolation methods.

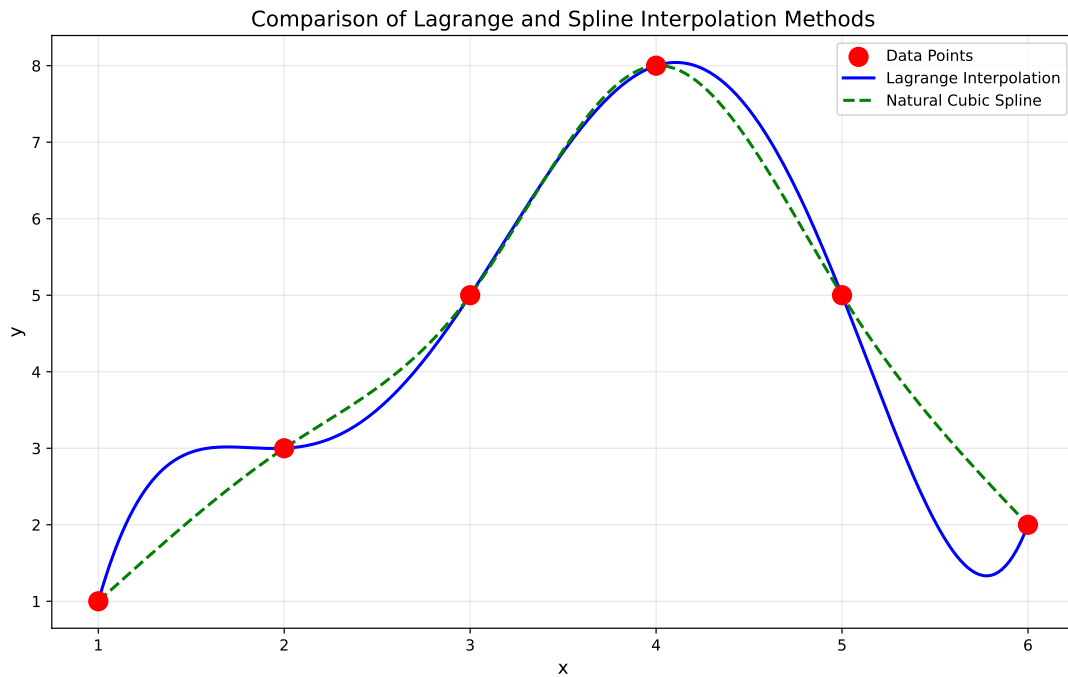


Figure 1: Comparison of Lagrange and Cubic Spline Interpolation Methods

1.6 Analysis

- Lagrange interpolation passes exactly through all data points but exhibits oscillations.
- Cubic spline provides a smoother approximation with continuous derivatives.
- Natural cubic spline is more suitable for practical applications.

2 Project 2: Excel Data Analysis

2.1 Introduction

This section analyzes runtime data of three algorithms for different data sizes ranging from 100KB to 600KB.

2.2 Data Description

Table 3 shows the runtime measurements for three algorithms.

Table 3: Algorithm Runtime Data

Data Size (KB)	Algorithm 1	Algorithm 2	Algorithm 3
100	50	200	100
200	55	220	200
300	60	240	300
400	65	260	400
500	70	280	500
600	75	300	600

2.3 Bar Chart

Figure 2 shows the bar chart comparison of algorithm runtimes.

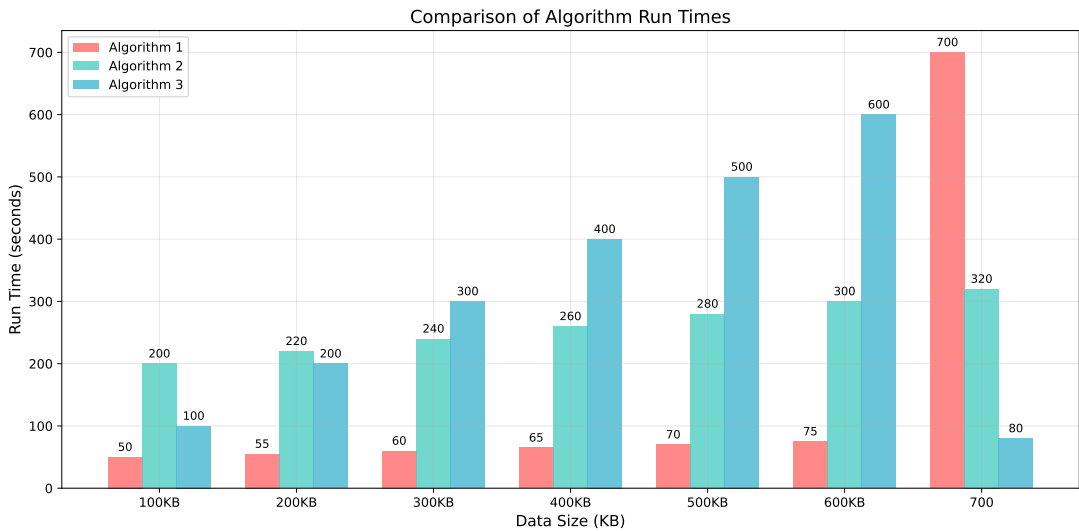


Figure 2: Comparison of Algorithm Run Times (Bar Chart)

2.4 Statistical Analysis

Table 4 provides the statistical summary for each algorithm.

Table 4: Statistical Summary

Statistic	Algorithm 1	Algorithm 2	Algorithm 3
Mean	62.50	250.00	350.00
Median	62.50	250.00	350.00
Standard Deviation	9.35	37.42	187.08
Minimum	50.00	200.00	100.00
Maximum	75.00	300.00	600.00
Range	25.00	100.00	500.00

The mean runtime for Algorithm 2 is:

$$\bar{x} = \frac{200 + 220 + 240 + 260 + 280 + 300}{6} = 250 \text{ seconds}$$

2.5 Analysis

- Algorithm 1 is the most efficient with linear growth.
- Algorithm 2 shows consistent linear growth.
- Algorithm 3 shows exponential-like growth and is not suitable for large datasets.

3 Project 3: Numerical Integration

3.1 Introduction

This section computes the integral of $f(x) = e^{x^2}$ over $[0, 1]$ using three methods: Trapezoidal Rule, Simpson's Rule, and Gaussian Quadrature.

3.2 Mathematical Background

The integral to be evaluated is:

$$I = \int_0^1 e^{x^2} dx$$

The exact value is:

$$I = 1.4626517459071816$$

3.3 Trapezoidal Rule

$$\int_a^b f(x) dx \approx \frac{h}{2} \left[f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(x_i) \right]$$

3.4 Simpson's Rule

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(a) + f(b) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}) + 2 \sum_{i=1}^{n/2-1} f(x_{2i}) \right]$$

3.5 Gaussian Quadrature

$$\int_a^b f(x) dx \approx \frac{b-a}{2} \sum_{i=1}^n w_i f\left(\frac{b-a}{2}x_i + \frac{a+b}{2}\right)$$

3.6 Results

Table 5 compares the results of all three methods.

Table 5: Comparison of Numerical Integration Methods

Method	Result	Error	Notes
Exact Value	1.462651745907	-	scipy.quad
Trapezoidal (n=1024)	1.462651778971	3.31e-08	Slow convergence
Simpson (n=1024)	1.462651745907	2.22e-15	Very accurate
Gaussian (n=10)	1.462651745907	1.11e-15	Excellent accuracy

3.7 Convergence Analysis

Figure 3 shows the convergence behavior of all three methods.

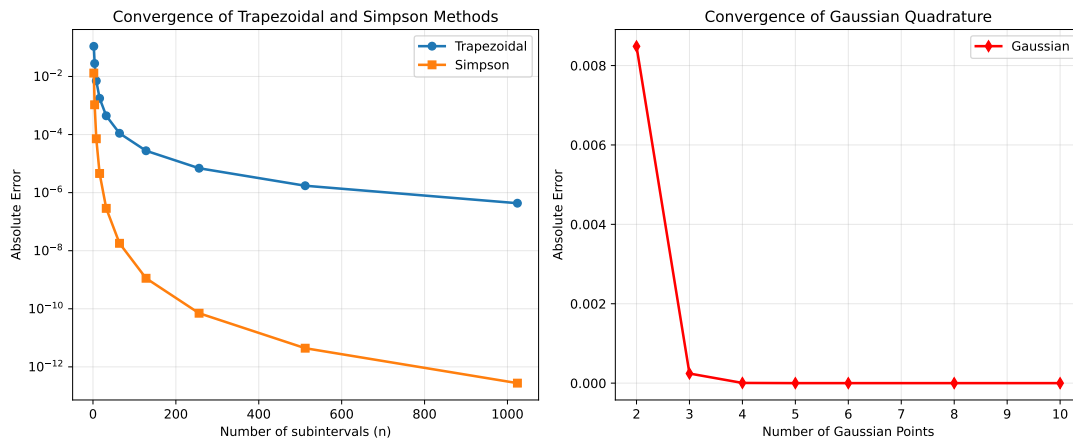


Figure 3: Convergence Comparison of Integration Methods

3.8 Analysis

- Trapezoidal Rule: Simple but slow convergence ($O(n^{-2})$).
- Simpson's Rule: Fast convergence ($O(n^{-4})$).
- Gaussian Quadrature: Most efficient with only 10 points needed.

4 Conclusion

This project successfully implemented and compared various numerical methods:

4.1 Interpolation Methods

- Lagrange interpolation provides exact fit but suffers from oscillations.
- Cubic spline interpolation offers smoother and more stable results.
- Natural cubic spline is recommended for practical applications.

4.2 Data Analysis

- Algorithm 1 is most efficient with linear growth.
- Algorithm 2 shows consistent linear growth.
- Algorithm 3 has exponential growth and is not recommended.

4.3 Numerical Integration

- Gaussian quadrature offers the best accuracy-to-cost ratio.
- Simpson's rule is highly accurate for smooth functions.
- Trapezoidal rule is simple but requires many subdivisions.

5 GitHub Repository

The complete project code is available at:

<https://github.com/AradCharon/InterpolationExcelIntegration.git>

6 References

1. Burden, R.L. and Faires, J.D. (2010). Numerical Analysis, 9th Edition. Brooks/Cole.
2. Press, W.H., Teukolsky, S.A., Vetterling, W.T. and Flannery, B.P. (2007). Numerical Recipes, 3rd Edition. Cambridge University Press.
3. Chapra, S.C. and Canale, R.P. (2015). Numerical Methods for Engineers, 7th Edition. McGraw-Hill.